Voice Scribe v2–v3
Local-First Speech-to-Text for Development Workflows

Johnraymond Yanez

Voice Scribe

April 5, 2026

Overview

- **Motivation** — Why local transcription for developer workflows?
- **Architecture** — Two-service system over localhost
- **Container Strategy** — GPU passthrough, model caching, isolation
- **Audio Pipeline** — Browser recording through inference
- **Frontend** — React component tree and state machine
- **Security** — Threat model, privacy, operational telemetry
- **Development** — Workflow, tooling, and testing
- **Roadmap, v2.0+** — Milestones and what just shipped
- **v3.0** — Desktop application, sidecar lifecycle, triage

The Problem

Developer workflows increasingly involve AI coding assistants — tools where natural language is the primary input.

Friction points

- Typing long prompts breaks flow
- Context-switching between thinking and typing
- Cloud transcription raises privacy concerns
- Existing tools not optimized for dev use cases

What we want

- Speak naturally, get text
- Text stays on your machine
- One-click copy into any tool
- No accounts, no cloud, no latency

Core Insight

Speech is the fastest way to externalize a thought. The bottleneck is getting that speech into text *locally* with sufficient accuracy.

Design Principles

Four principles govern every decision:

1. **Local-first**
   Audio and transcripts stay on the user's machine. No cloud services required at any point.
2. **Separation of UI and inference**
   The UI layer owns mic access, clipboard, and editing. The inference layer owns model loading and GPU inference.
3. **Audio is transient, text is durable**
   Raw audio is short-lived input — deleted after processing. The transcript is the user-facing artifact.
4. **Scope matches the milestone**
   Repo structure and services reflect current reality, not future aspirations. Architecture stays flat until complexity is earned.

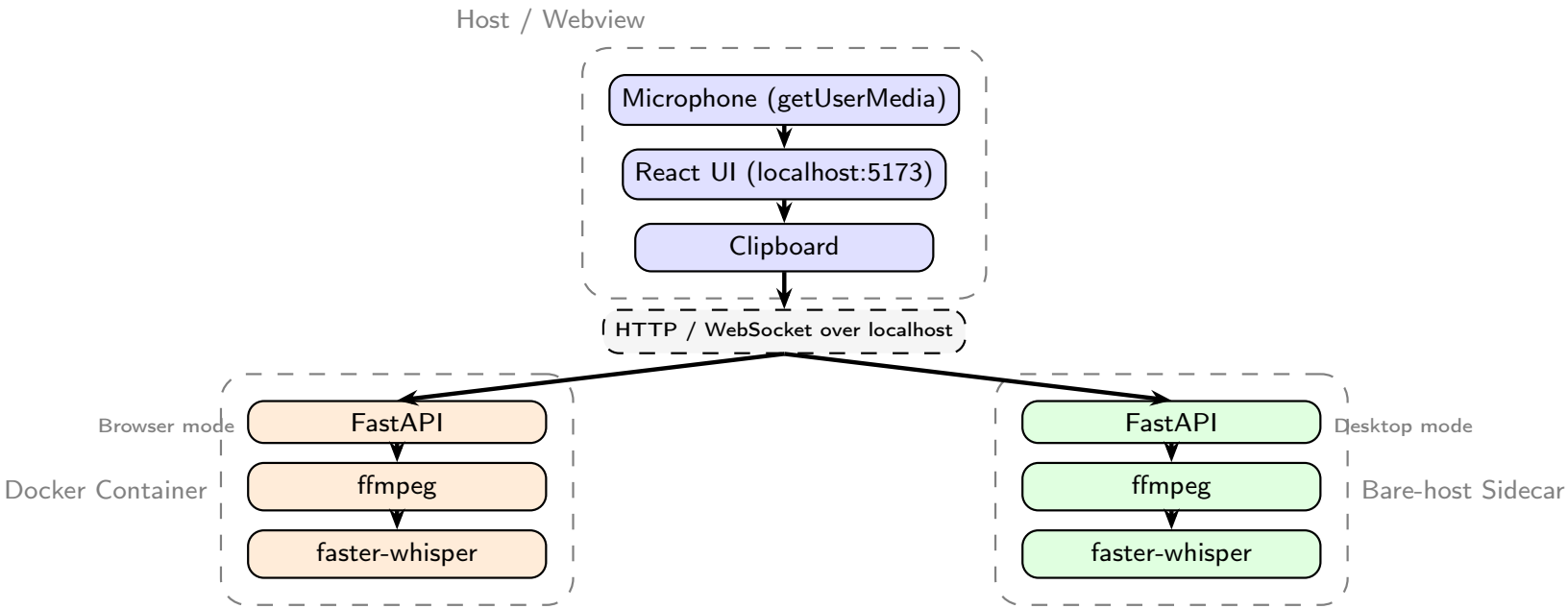
System Architecture

Voice Scribe is a two-service system communicating over localhost.

- The **frontend** runs on the host — it owns browser APIs (microphone, clipboard, DOM) that cannot exist inside a container
- The **backend** runs GPU-accelerated inference via CUDA. Two runtime contexts: Docker container (browser mode) or bare-host Python sidecar (desktop/Tauri mode)
- Communication uses WebSocket (default, v2.1+) and HTTP (fallback). No reverse proxy, no message queue
- Both services bind to `localhost` only — never network-accessible

The split is dictated by hardware constraints, not architectural preference. Each service does what only it can do.

System Architecture — Diagram



Same backend, different isolation boundary. Browser mode uses Docker; desktop mode uses a Tauri-managed sidecar. Both bind to `localhost:8000`.

Tech Stack

Layer	Technology	Rationale
Frontend	React, TypeScript, Vite	Mic access, clipboard, fast iteration
Backend	Python, FastAPI	Model ecosystem, easy containerization
STT Engine	faster-whisper (large-v3)	Accuracy/speed on local GPU
Transcode	ffmpeg (in container)	Normalizes browser audio to WAV
Container	Docker, CUDA 12.6	GPU isolation, reproducibility
Desktop	Tauri v2, Rust	Native OS access, sidecar management

Selection Principle

Each layer is selected for its ecosystem strength at that boundary.

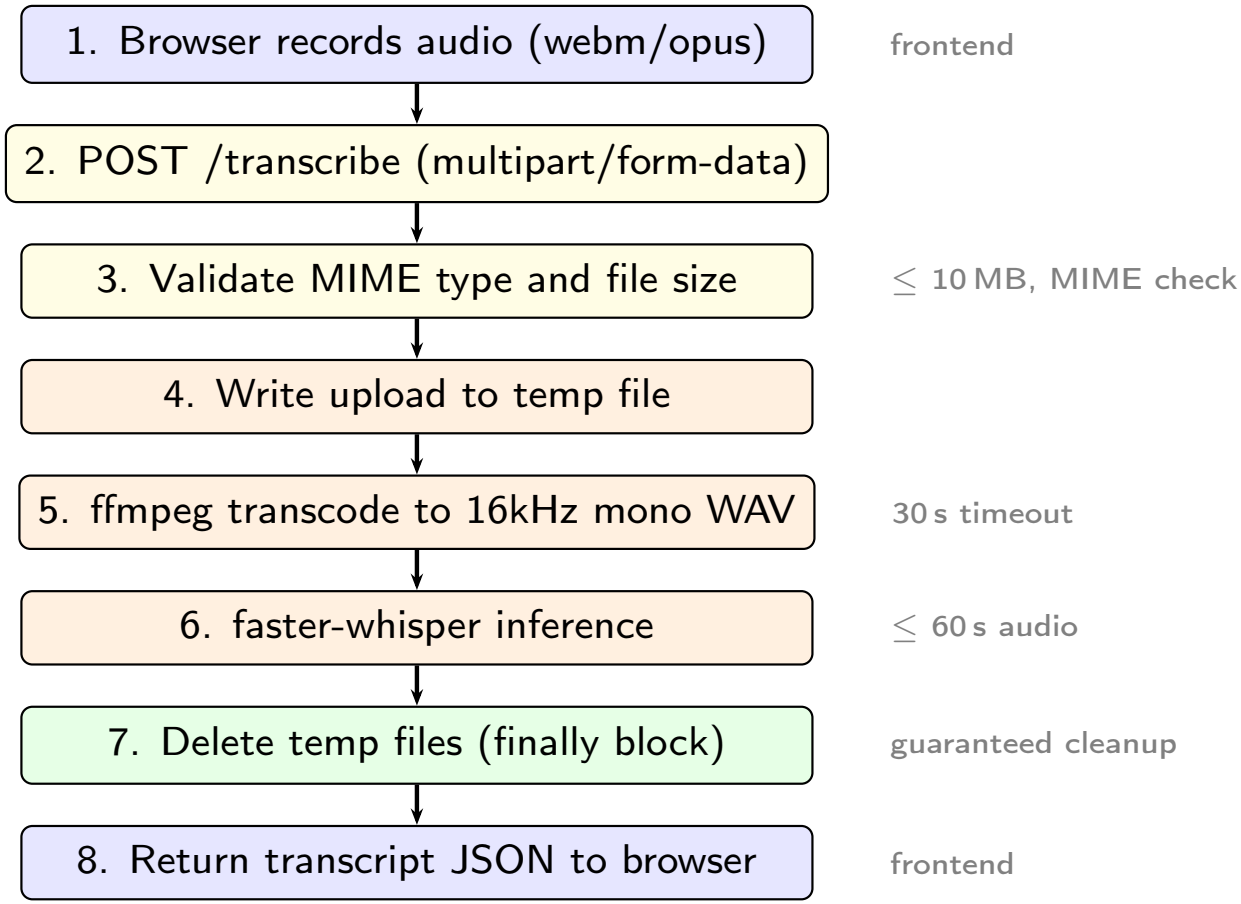
Request Lifecycle

A single transcription request passes through eight stages. The frontend handles stages 1 and 8; the backend handles 2–7.

- **Validation** (stages 2–3) rejects bad input early — before any expensive I/O or computation
- **Processing** (stages 4–6) is the core pipeline: temp file, transcode, inference
- **Cleanup** (stage 7) is guaranteed by `try/finally` — both temp files are deleted on success *and* failure
- Every stage between 3–7 is bounded by explicit limits: 10 MB upload, 60 s duration, 30 s ffmpeg timeout

Note : The synchronous design means the HTTP connection stays open during inference. For v1's single-user use case, this is correct.

Request Lifecycle — Diagram



Blue = frontend, Yellow = validation, Orange = processing, Green = cleanup.

Host vs. Container vs. Sidecar

Two runtime modes share the same UI/inference boundary.

UI Layer
(browser or webview)

- Microphone access
- Audio recording
- UI rendering
- Text editing
- Clipboard access

Docker Container
(browser mode)

- Model weights
- ffmpeg transcode
- GPU inference
- MIME validation
- Temp file lifecycle

Bare-host Sidecar
(desktop mode)

- Same five responsibilities
- Tauri manages lifecycle
- CUDA library discovery
- WebKitGTK permissions

Backend code is identical — only the process boundary changes. Desktop mode adds: sidecar spawn, health poll, CUDA runtime injection.

GPU Passthrough

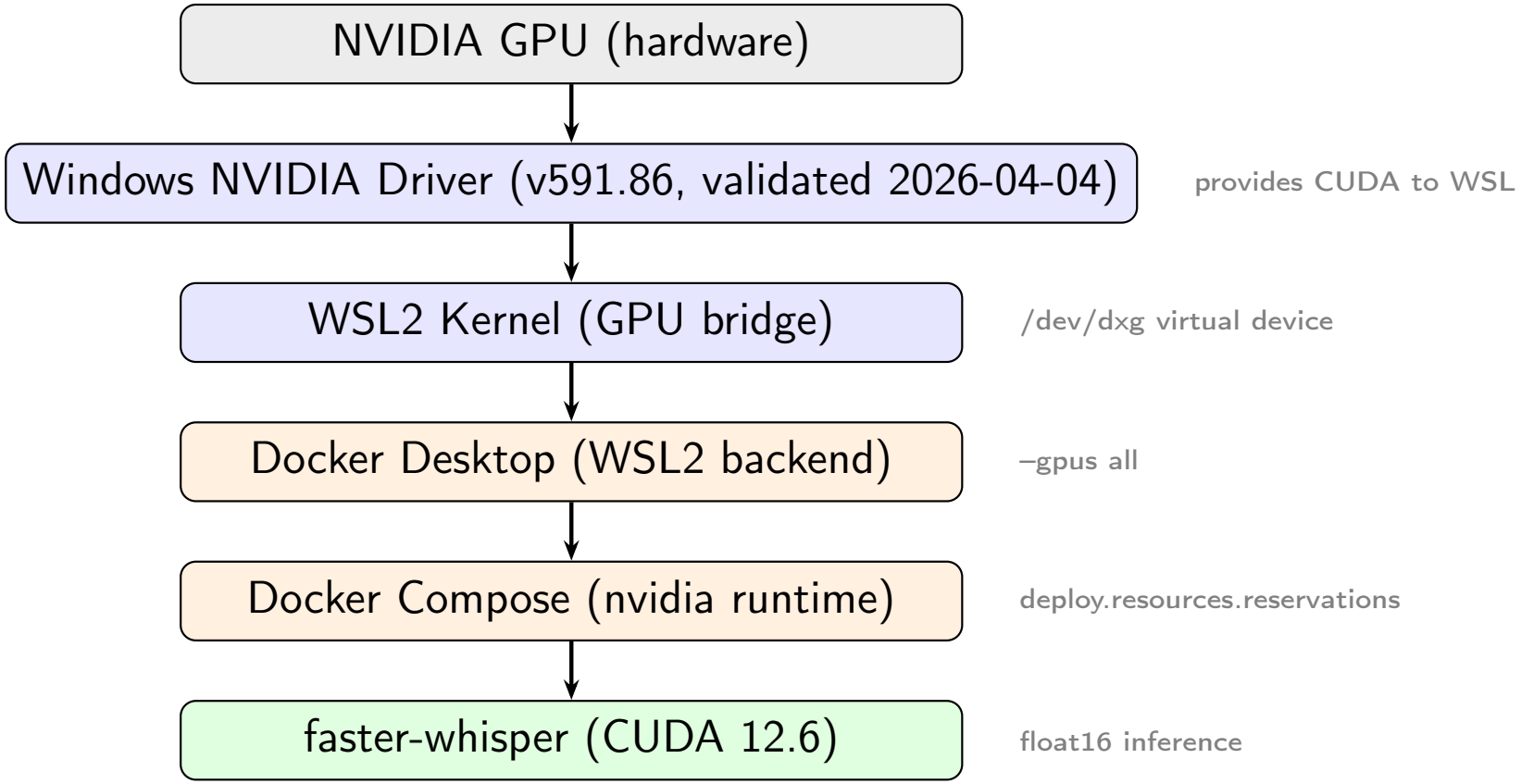
Voice Scribe uses NVIDIA GPU passthrough from Windows through four layers. Each layer must be correctly configured for the next to function.

- **Windows NVIDIA driver** provides CUDA to the WSL2 kernel
- **WSL2 kernel** bridges GPU access into Linux userspace
- **Docker Desktop** (WSL2 backend) exposes the GPU to containers
- **Docker Compose** `deploy.resources` reserves the device

This is the highest-risk setup step in the entire project. A misaligned driver version, WSL kernel, or Docker Desktop setting breaks the chain silently.

Note : The bootstrap script `ops/scripts/bootstrap_gpu_check.sh` gates all development behind GPU verification. Do not write application code until it passes.

GPU Passthrough — Diagram



Each layer depends on the one above it. The bootstrap check verifies the full chain by running `nvidia-smi` inside a GPU-enabled container.

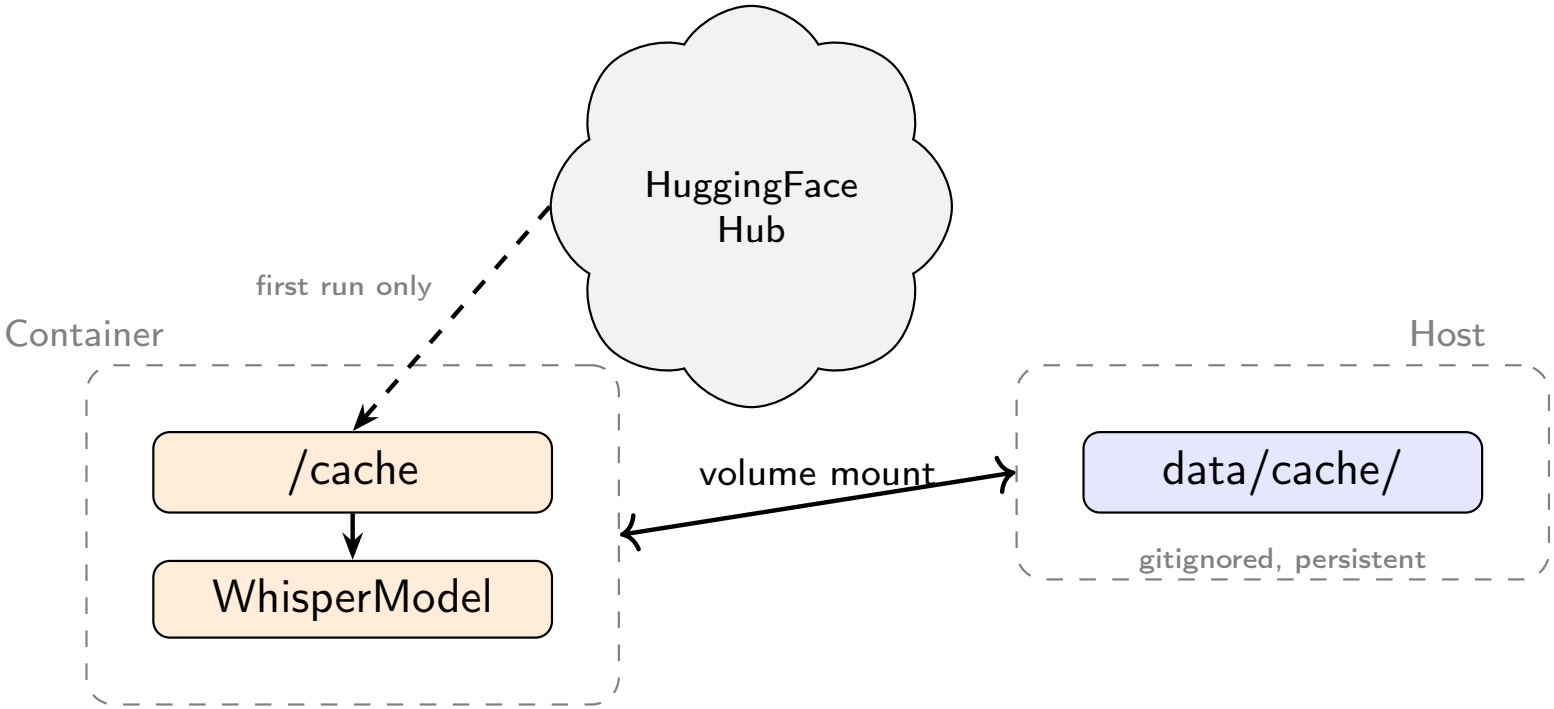
Model Caching

The `large-v3` model is ∼3 GB. It downloads once from HuggingFace and must persist across container restarts.

- Model weights are **not** baked into the Docker image — this would bloat the image and prevent model swaps
- A host-mounted volume (`data/cache/`) maps to `/cache` inside the container
- First run: ∼60 s download. Subsequent runs: < 2 s load
- `data/cache/` is gitignored — weights never enter the repo

Remark : The compose file sets `user: ${UID}:${GID}` so cache files are owned by the host user, avoiding permission mismatches between container and host filesystems.

Model Caching — Diagram



Dashed arrow = one-time download. Bidirectional arrow = volume mount. Model loads from host filesystem on every startup after first run.

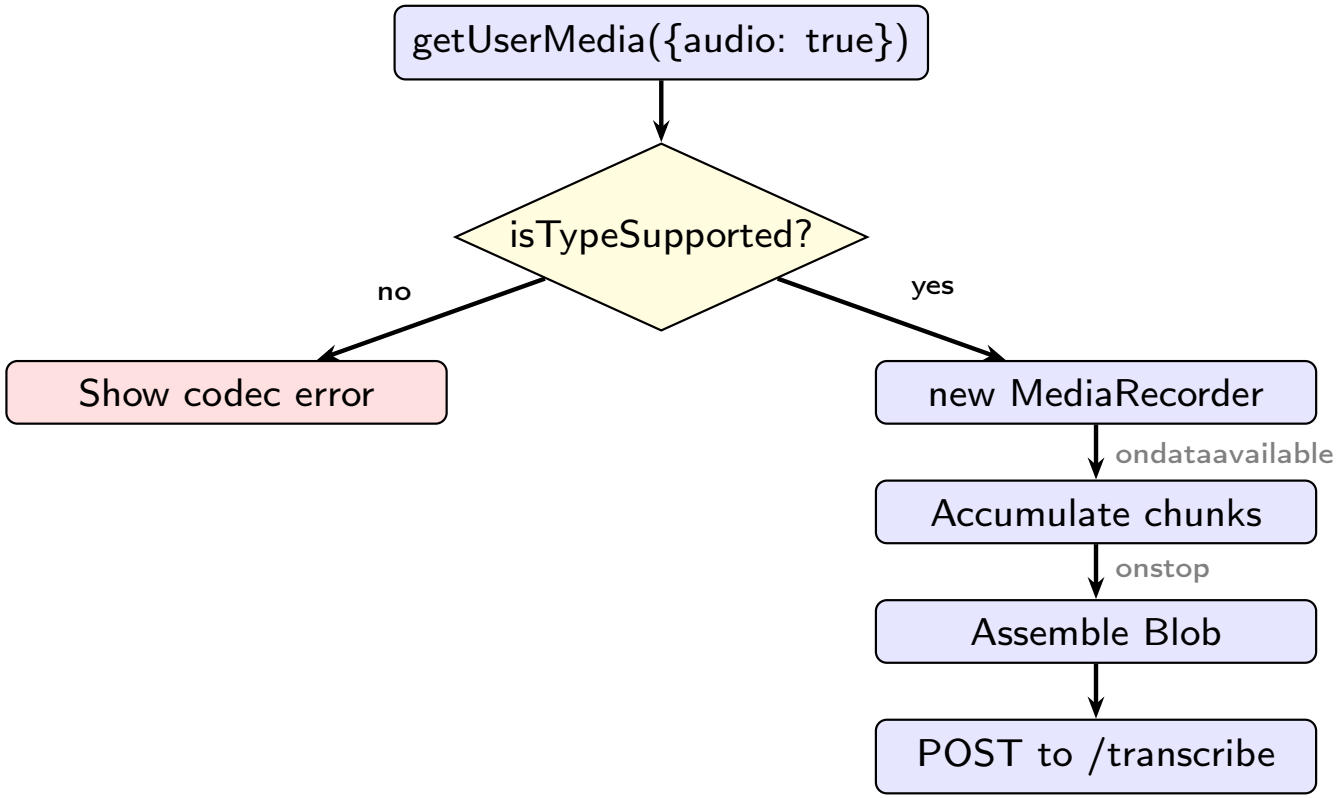
Browser Recording

Audio capture uses the Web `MediaRecorder` API, which provides raw access to the microphone through the browser.

- The recorder explicitly requests `audio/webm;codecs=opus` — never relies on browser defaults
- A pre-flight check via `isTypeSupported()` runs before any recording attempt. If the codec is unsupported, the user sees a clear error message
- Audio chunks accumulate in memory during recording and are assembled into a single Blob on stop
- Media stream tracks are cleaned up on unmount to release the microphone

Note : `MediaRecorder` does not guarantee `webm/opus` across all browsers. Chrome on Windows is the supported browser for v1. We fail explicitly on unsupported browsers rather than producing corrupt recordings.

Browser Recording — Diagram



The diamond is a hard gate — recording never starts if the codec is unsupported. This prevents silent failures where the browser records in an unexpected format that the backend cannot transcode.

Server-Side Transcoding

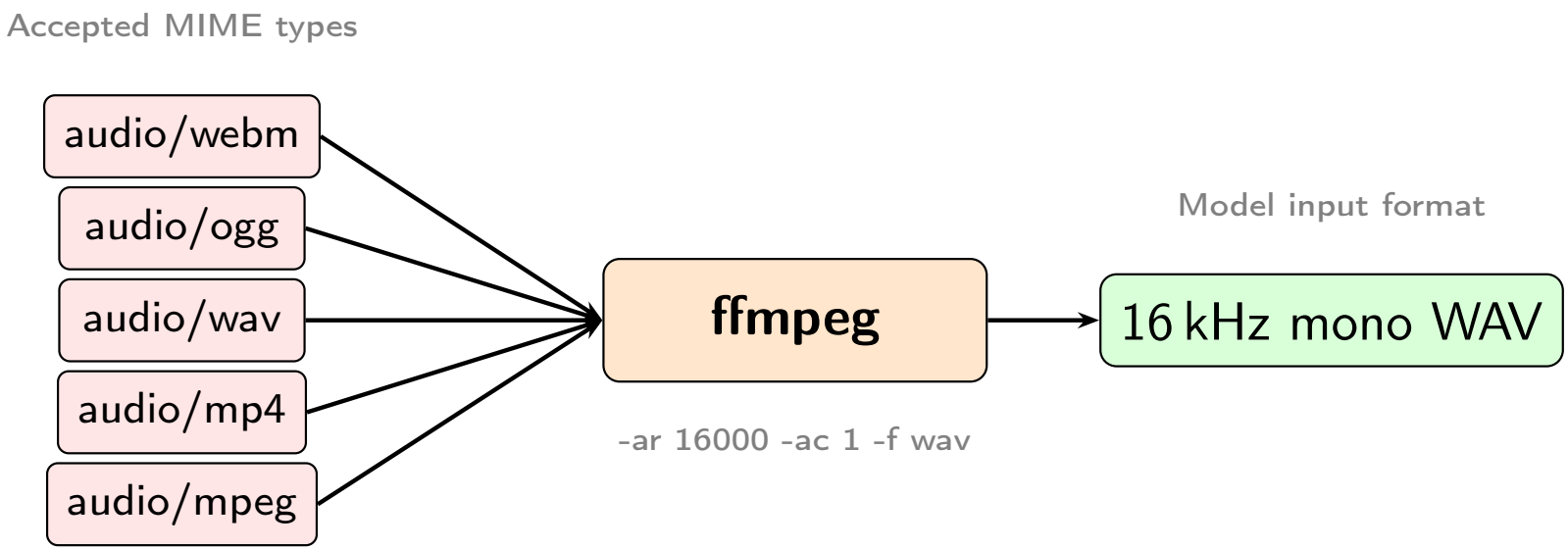
The browser produces `webm/opus`. The model expects 16 kHz mono WAV. Transcoding is performed server-side by ffmpeg inside the container.

- The backend accepts five audio MIME types — any format ffmpeg can decode is a valid input
- All inputs are normalized to the same output: 16 kHz sample rate, single channel, PCM WAV
- Adding a new input format requires only adding the MIME type to the allowlist — zero frontend changes
- ffmpeg runs with a 30-second timeout to prevent hung processes

Why Not Transcode in the Browser?

Keeping transcoding server-side means the frontend is format-agnostic. It sends whatever the `MediaRecorder` produces, and the backend normalizes. This isolates codec complexity to one place.

Server-Side Transcoding — Diagram



Five input formats funnel through a single normalization step. The model always receives identical input regardless of browser or source format. Safety bounds: 10 MB cap, 60 s duration, 30 s timeout.

Inference

`faster-whisper` provides CTranslate2-optimized Whisper inference, running locally on the GPU with no external API calls.

Model configuration

- Model: `large-v3` (default)
- Device: CUDA (GPU)
- Compute type: `float16`
- Beam size: 5
- Fallback: `small` on CPU, `int8` *(verified)*

Runtime behavior

- Loaded at startup, held in GPU memory
- Swappable at runtime (v2.0)
- Synchronous (blocks thread)
- Single-user — no concurrency
- VAD filtering enabled

Synchronous design: `faster-whisper` blocks the thread during inference. Correct for single-user. Rework path: `asyncio.to_thread()` or background task with polling (both change the API contract).

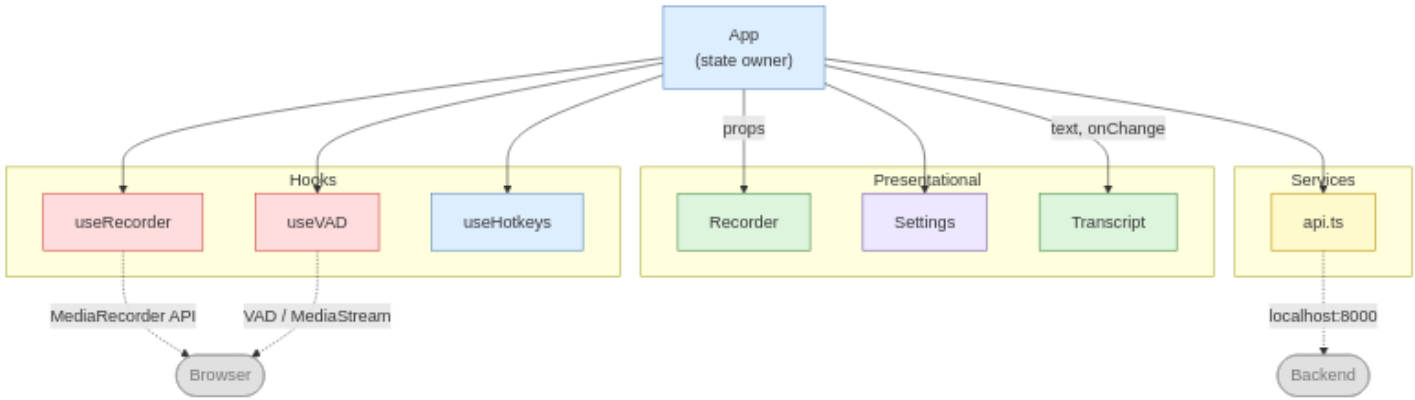
Component Architecture

The frontend is a small React application with a clear separation between container logic, presentational components, and services.

- **App** is the sole container component — it owns all state, calls `useRecorder` and `useHotkeys` directly, and orchestrates the full flow
- **Recorder**, **Transcript**, and **Settings** are presentational — they receive props and emit callbacks
- **useRecorder** and **useHotkeys** are custom hooks consumed by App, keeping browser API complexity out of components
- **api.ts** is a thin service layer — typed API calls for transcription, model list, model swap, and WebSocket streaming

Remark : No routing, no global state library, no build-time environment variables. Complexity is earned.

Component Architecture — Diagram



Blue = state owner / wiring (App, useHotkeys) Red = active-mic hooks (useRecorder, useVAD) Green = presentational components Lavender = settings overlay Yellow = service layer (api.ts) Gray = external systems. Dashed arrows = external system boundaries.

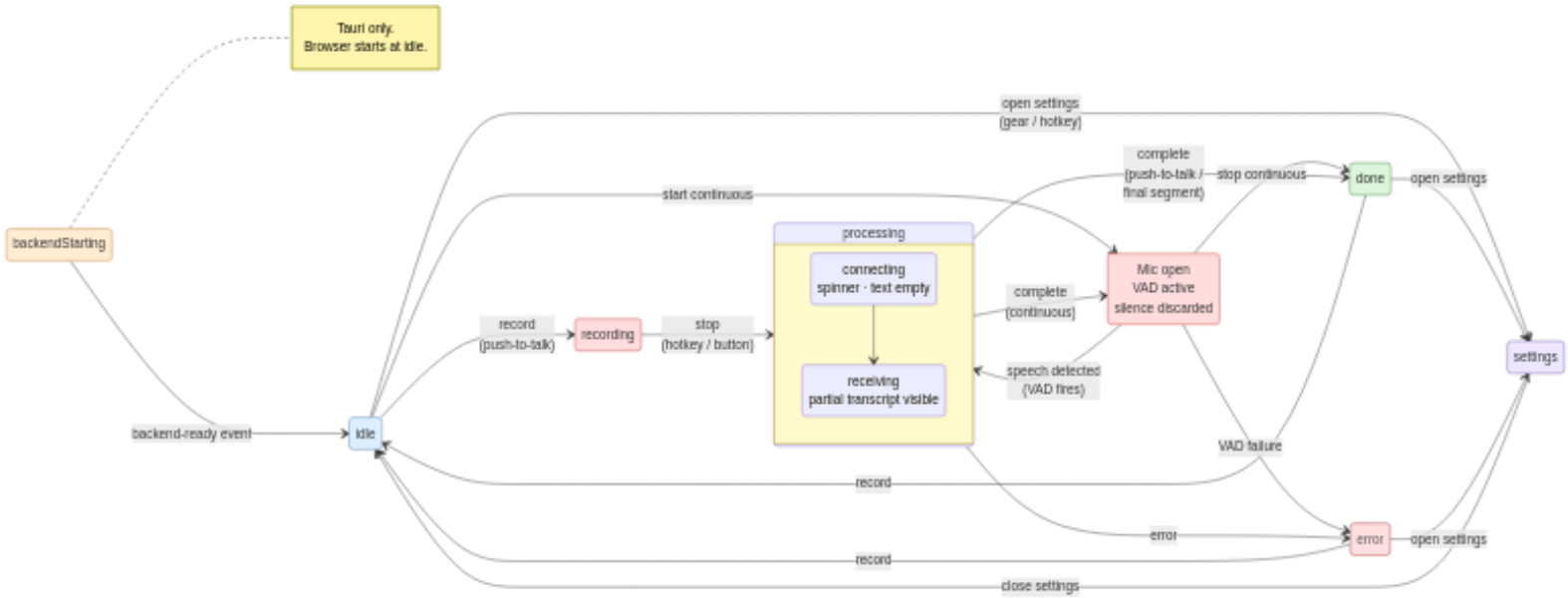
State Machine

The UI has six recording pipeline states plus a settings overlay.

- **idle** — Ready. Record button enabled.
- **recording** — Push-to-talk: mic active. Button shows "Stop".
- **listening** — Continuous: mic open, VAD active, silence discarded.
- **processing** — Transcription in progress. Sub-phases: *connecting* (spinner) and *receiving* (partial transcript). In continuous mode, Stop remains active for drain-then-done.
- **done** — Transcript ready for review and copy.
- **error** — Error message displayed. User can retry.

Settings overlay is orthogonal — a separate `settingsOpen: boolean`, not a seventh state. Accessible from idle, done, and error only.

State Machine — Diagram



Blue = idle Red = mic active (recording / listening) Yellow = in-flight (processing) Green = done Pink = error Lavender = settings overlay (`settingsOpen: boolean`, orthogonal to pipeline). Mode-conditional exit from `processing`: push-to-talk → `done`; continuous → `listening`.

Threat Model

Asset	Sensitivity	Location
Microphone audio	High	Browser memory, temp file (transient)
Transcript text	High	Browser memory, HTTP response
Session metadata	Low	Container logs (durations, formats)
Model weights	None	data/cache/ (public files)

Primary attack surfaces and mitigations:

- **Upload endpoint** — MIME validation, 10 MB size cap, 60 s duration cap, 30 s ffmpeg timeout
- **Temp storage** — `try/finally` cleanup; dedicated directory; crash residuals monitored
- **CORS / origin** — Restricted to localhost origins only; no wildcard
- **Dependencies** — Pinned versions, weekly Dependabot scans

Note : Most likely failure: accidental LAN exposure via bind address change.

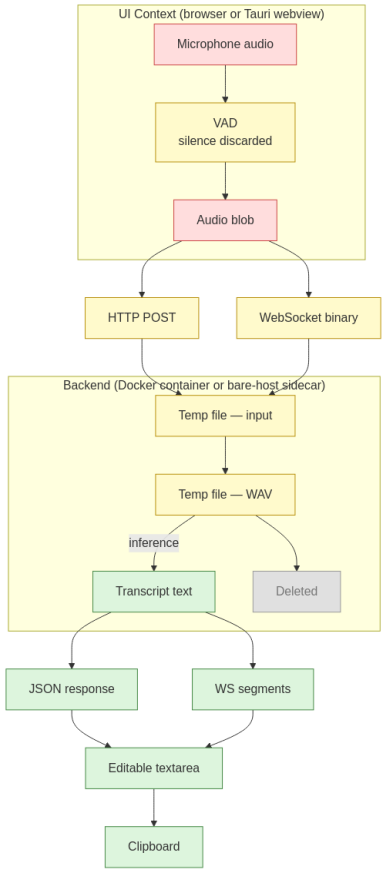
Privacy — Data Flow

- Audio exists in three places: browser memory, localhost transit (HTTP POST or WebSocket binary), and a temp file (seconds). Both transports are localhost only.
- Transcript exists in two places: localhost transit (JSON response or WebSocket segment stream) and the browser textarea. Retention is identical for both transport paths.
- **Nothing is persisted server-side** — API is stateless between requests.
- Logs contain only durations, formats, and detected language — never content.

Network Boundary

API binds to `localhost:8000`. CORS restricted to localhost origins. WebSocket origin enforced in handler (CORS middleware does not cover WS). Zero external network traffic.

Privacy — Data Flow Diagram



Red = sensitive audio (transient). Yellow = ephemeral intermediate. Green = transcript output (user-controlled). Gray = deleted. Both transports are localhost only.

Operational Telemetry

The system logs *operations*, not *content*. Every event includes an ISO 8601 timestamp via `structlog`.

Event	Fields
`model download started`	model name, device
`model download complete`	duration
`transcode started`	input MIME type
`transcode complete`	output format, duration
`inference started`	audio file path
`inference complete`	duration, detected language
`model swap started`	previous model, new model
`model swap complete`	duration

- All output is structured and greppable from container logs
- Log level defaults to `info`; adjustable via `LOG_LEVEL`
- Transcript content is **never** logged at any level — this is a hard requirement, not a convention

Workflow & Tooling

Development environment: WSL2 Linux filesystem, VS Code with WSL extension, Docker Desktop with WSL2 backend, Chrome for testing.

Makefile targets:

Command	Purpose
`make dev`	Start API container + frontend dev server
`make build`	Build the API Docker image
`make test`	Run all tests (pytest + vitest)
`make gpu-check`	Verify GPU/Docker readiness
`make desktop`	Start Tauri desktop app + sidecar

Smoke test (fastest full-stack feedback loop):

```
bash ops/scripts/smoke_test.sh ops/scripts/test.mp3
```

Tests `GET /health`, `POST /transcribe`, and `GET /models` against the running API.

Testing Strategy

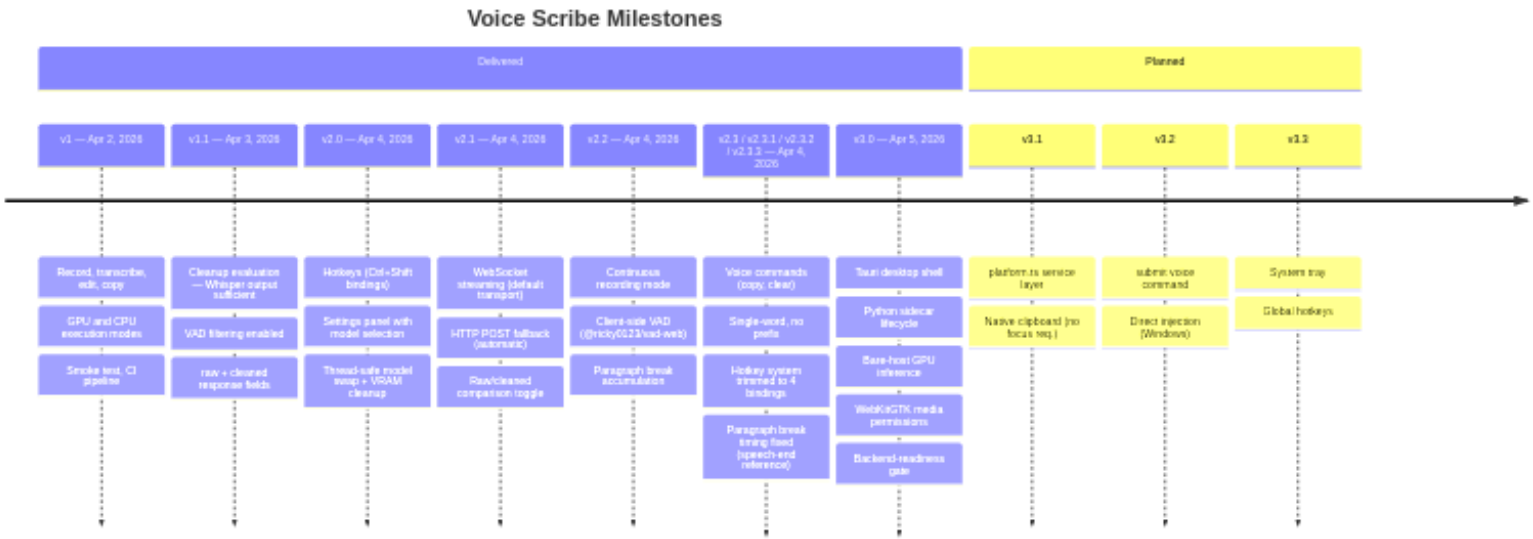
Backend (pytest, 41 tests)

- Health, MIME validation, size limits, happy-path transcription
- Transcript cleanup: normalization, idempotency (parametrized)
- Model list and swap endpoints
- WebSocket: origin enforcement, payload validation, happy path, inference failure, swap concurrency contract

Frontend (vitest, 70 tests)

- Recorder and Transcript (v1 suite)
- Settings panel: swap states, comparison toggle, recording mode
- VAD hook and WAV encoder: lifecycle, callbacks; header, magic bytes, clamp
- App and voice command: continuous cycle, drain-then-done; normalization, exact-segment guard, false positives

Milestone Timeline



Delivered milestones show actual dates and duration. Planned milestones show sequence only — no hard deadlines. The project iterates as it continues; scope is earned, not scheduled.

Roadmap Detail

v2.0 (complete)

- Hotkey system
- Settings panel (model selection)
- State machine diagram update

v2.1 (complete)

- WebSocket streaming (default transport)
- HTTP POST fallback (automatic)
- Raw/cleaned comparison toggle

v2.2 (complete)

- Continuous recording mode
- Client-side VAD (`@ricky0123/vad-web`)
- Paragraph break accumulation

v2.3.x (complete)

- Voice commands: copy, clear (single-word, no prefix)
- Hotkey system trimmed to 4 bindings
- Paragraph break timing fixed (v2.3.3)

v3.0 (complete) — Tauri shell + sidecar. v3.1: native clipboard; v3.2: `submit` injection; v3.3: tray + global hotkeys.

v2.0 — Hotkeys + Settings

Hotkey system — keyboard shortcuts for recording control, copy, clear, and settings toggle. All use `Ctrl+Shift+<key>` modifier to avoid conflicts with the editable textarea and Chrome built-in bindings.

Hotkey	Action
`Ctrl+Shift+Space`	Toggle recording
`Ctrl+Shift+Y`	Copy transcript
`Ctrl+Shift+X`	Clear transcript
`Ctrl+Shift+S`	Toggle settings panel

Settings panel — runtime model selection via `GET /models` and `POST /models/swap`.

- Threading lock + VRAM cleanup on swap; old model stays loaded if swap fails
- Regular `def` endpoint — thread pool, event loop alive during reload
- Models limited to `WHISPER_AVAILABLE_MODELS`; no auto-discovery

v2.1 — WebSocket Streaming

Recording flow unchanged — start, speak, stop. What changes is delivery.

- Client connects to `ws://localhost:8000/ws/transcribe`; full audio blob sent as binary after stop
- Server streams `{"type":"segment"}` messages as faster-whisper yields each segment; `{"type":"complete"}` closes with full metadata
- Partial text accumulates in the textarea during inference
- `ws.onerror` falls back to `POST /transcribe`; `type:"error"` surfaces immediately — no fallback
- Swap-safe: inference captures a strong model reference at thread start; VRAM cleanup defers until completion

Note on GPU hardware

RTX 3090 + `large-v3`: 30 s audio finishes in ≈2.5 s — segments arrive in a burst. Streaming is perceptible on CPU or in v2.2 continuous mode.

v2.2 — Continuous Recording Mode

Mic stays open; browser VAD decides when speech ends. Push-to-talk is unchanged.

VAD & recording

- `@ricky0123/vad-web`, Silero VAD v4, ONNX/WASM — runs in-browser
- Silence discarded before any network round-trip
- Each utterance is an independent WebSocket round-trip; results accumulate in order
- No backend changes required
- AudioWorklet off-main-thread; `onSpeechEnd` delivers 16 kHz mono PCM

Transcript behavior

- **Paragraph breaks** — silence gap > 3 s between utterances inserts `\n\n`
- **Drain-then-done** — stop during processing completes the in-flight segment before transitioning to *done*

v2.3.x — Voice Command Vocabulary

A small control language embedded in the voice stream. Commands are exact-segment matches: the *entire* transcribed segment must be the command phrase. Mid-sentence occurrences ("let me clear this up") do not fire.

Word	Action
`copy`	Copy transcript to clipboard
`clear`	Clear transcript buffer

Single-word design — no prefix. VAD-split is structurally impossible with one word; two-word phrases split at the natural inter-word pause and neither fragment matches. Gate 1 confirmed 10/10 copy, 9/10 clear on `large-v3`.

v2.3.x — Detection Architecture

Detection surface: `onComplete` **only** — both modes.

- **Normalization** — trim, lowercase, strip trailing `.!?,`. Handles Whisper's capitalized first letter and trailing period.
- **Exact-segment match** — `COMMAND_MAP` lookup in `voiceCommands.ts`. Adding or removing a command is one table entry.
- **Command segments are consumed** — nothing appended to the transcript. No strip function needed.
- **clear** sets transcript state directly; does not call `handleClearTranscript`. Transition block owns `RecordingState`.
- **transcriptRef** — stable ref gives "copy" a consistent view of the transcript inside the callback closure.

v2.3.x — Known Limitations

Browser sandbox constrains delivery.

- **Recording is cross-window** — VAD holds mic access at the OS level. Voice Scribe listens in the background without focus.
- **Clipboard write requires focus** — `navigator.clipboard.writeText()` is blocked in background tabs. "copy" fires correctly; the write fails silently if focus is elsewhere.
- **No injection into other applications** — text cannot be delivered directly to a focused editor or terminal.

The gap: the tool listens hands-free; it cannot deliver hands-free. The complete loop requires native OS access.

v3.0 establishes the platform (Tauri shell + sidecar). **v3.1** delivers the native clipboard that closes the focus gap. **v3.2** adds direct text injection.

v3.0 — Desktop Application

v3.0 is a platform change, not a feature addition. All v2 functionality is preserved — recording modes, streaming, VAD, voice commands, settings.

What changed

- Tauri v2 desktop shell
- Python sidecar (replaces Docker)
- Bare-host GPU inference
- WebKitGTK media permissions
- CUDA runtime discovery
- Backend-readiness gate

What stayed the same

- Same React frontend build
- Same FastAPI backend code
- Same WebSocket/HTTP transports
- Same localhost-only binding
- Browser mode unchanged
- 111 tests (70 frontend, 41 backend)

Runtime split

`window.__TAURI__` at module scope. One Vite build serves both browser and Tauri webview. No build-time flags.

v3.0 — Sidecar Lifecycle

Tauri spawns the Python API as a managed child process. No Docker, no container overhead.

Spawn `find_uvicorn()` resolves the Python executable. `cuda_ld_library_path()` discovers pip-installed NVIDIA libraries under `site-packages/nvidia/*/lib/` and injects via `LD_LIBRARY_PATH`.

Health poll Background thread polls `GET http://127.0.0.1:8000/health` every 500 ms. 120 s timeout. Crash detection via `try_wait()` each cycle — port conflicts surface immediately, not after full timeout.

Ready On health success, emits `backend-ready` Tauri event. UI transitions from "Starting backend..." to active.

Shutdown `on_window_event` sends SIGTERM on `CloseRequested`. 5 s grace period, then SIGKILL. `Child` handle in `Mutex<Option<Child>>` as Tauri managed state.

v3.0 — Triage Highlights

Two architecturally significant findings during desktop bring-up.

WebKitGTK media permissions. Two-layer model for `getUserMedia`: (1) feature gate (`enable-media-stream`) and (2) runtime permission grant via the `permission-request` signal. Without both layers, recording is *silently denied* — no prompt, no error dialog. Chromium auto-prompts; WebKitGTK delegates to the application. Fix: `connect_permission_request` auto-grants all requests.

CUDA runtime discovery on bare host. `ctranslate2` dlopens `libcublas.so.12` at inference time. Docker's NVIDIA runtime provides it automatically. On bare host, it requires `pip install nvidia-cublas-cu12`. `cuda_ld_library_path()` discovers the library under `site-packages/nvidia/*/lib/` and injects it via `LD_LIBRARY_PATH` on the sidecar child process.

v3.0 — Backend-Readiness Gate

Problem: The sidecar takes 2–5 s to start (model loading). The UI must not allow recording until the backend is ready.

Tauri context

- `backendReady` starts `false`
- `useEffect` listens for `backend-ready` Tauri event
- Record button shows "Starting backend..."
- Event fires → button enables

Browser context

- `backendReady` starts `true`
- No gate, no behavior change
- Record button enabled on mount
- No Tauri dependency loaded

Detection: `typeof window.__TAURI__ !== "undefined"` at module scope. Two tests verify both paths (dynamic import with `vi.resetModules()` for the Tauri context).